Recurrent Networks: Time-Translation Equivariance as Architecture

An RNN is an MLP with a single prior wired into its weights: the same computation runs at every step of a sequence, and the new hidden state summarizes the past. The first move is identical to the CNN's: weight sharing across positions, where "position" is now an index in time rather than space.

This is the third post in a series on neural-network inductive biases. The CNN post introduced weight sharing across spatial positions, which gives translation equivariance for images. This post does the same move for sequences. The math, the implementation, and even the `+=` accumulation pattern in the backward pass all transfer directly. What changes is the *axis* the sharing runs along (time instead of space) and a new wrinkle the spatial version did not have: the layer's output at one timestep is also its input at the next, so the chain of computations forms a long sequential dependency. That sequential dependency is what makes RNNs hard to train at long horizons, and the post earns the right to name vanishing gradients as the price of the prior.

1. The prior

A sequence is an ordered series of inputs $x_1, x_2, \ldots, x_T$. The data type might be characters in a string, tokens in a sentence, ticks of a time series, frames of a video, or pixels in raster order. The defining structure is that *time matters*: x_t is closer to x_{t+1} than to x_{t+5}, and the same computation should usefully apply at any t.

What inductive bias commits to that structure? Two parts, the same shape as the CNN's:

- **Locality in time.** The prediction at t depends most heavily on the recent past, not on every token from the beginning of the sequence with equal weight.
- **Time-translation equivariance.** Shift the input sequence by Δt and the output shifts by the same Δt. The same computation applies at every step, so the model's weights do not depend on the absolute timestamp.

An MLP applied to a flattened sequence has neither prior. It treats x_t at every t as an independent feature, with separate weights for each position. It can learn the right thing from enough data, but it will not generalize across timesteps it has not seen. (You can imagine running the CNN post's shapes experiment on temporal sequences: the MLP that saw "the cat" at positions 1-3 fails to recognize "the cat" at positions 5-7.)

A 1-D convolution along the time axis already encodes both priors: shared filter weights at every position give translation equivariance, and a small filter size gives locality. That is a perfectly good architecture, sometimes called a fixed-context model, and a sister post in this series visits it directly.

The RNN makes a different commitment. Instead of looking at a *fixed window* of recent inputs, it carries a hidden state vector h_t that *summarizes the past*. The next state h_{t+1} is computed from the current state h_t and the current input x_{t+1}, using the same weights at every step. In principle this lets the model attend to arbitrarily distant past tokens through the state. In practice it has hard limits, which section 5 walks through.

2. The cell

A vanilla RNN cell is

$$h_t = \tanh(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b_h),$$

where W_{xh} is the input-to-hidden weight matrix, W_{hh} is the recurrent (hidden-to-hidden) weight matrix, b_h is the bias, and tanh is applied componentwise to the pre-activation vector. The cell's output is h_t itself; a separate `Linear` layer projects h_t to logits or any other target shape downstream.

The crucial property is that W_{xh}, W_{hh}, and b_h do not depend on t. The same weights compute the state update at every step. This is weight sharing across the time axis, exactly analogous to the CNN's weight sharing across spatial positions.

In code, the cell extends the `Layer` protocol for stateful layers: `forward(x, state)` takes an optional previous state and returns `(output, new_state)`; `backward(g, dstate_next)` takes an optional gradient on the new state and returns `(dL/dx, dL/dstate_prev)`. Stateless layers (`Linear`, `Tanh`, …) need no change. The two protocols share `parameters()` but have different `forward`/`backward` shapes, which is the same Module-vs-Functional split that real frameworks navigate.

```
class RNNCell(Layer):
    def __init__(self, input_size, hidden_size):
        # W_xh, W_hh, b_h with their gradient accumulators.
        # Stored as list[list[float]], one row per hidden unit,
        # matching Linear's structure so parameters() yields one
        # (values, grads) pair per row.
        ...

    def forward(self, x, state=None):
        if state is None:
            state = [0.0] * self.hidden_size
        # h_new = tanh(W_xh @ x + W_hh @ state + b_h)
        # Cache (x, state, h_new) for backward.
        ...
        return h_new, h_new

    def backward(self, dh_out, dstate_next=None):
        # Pop the cache for this timestep.
        # Total gradient on h_new = dh_out + dstate_next.
        # Apply tanh derivative, accumulate weight gradients, return
        # (dx, dh_prev).
        ...
        return dx, dh_prev
```

3. Time-translation equivariance

The argument from the CNN post transfers verbatim. Replace shift in space with shift in time.

Suppose we shift the input sequence by Δt, so the new sequence is $x'_t = x_{t+\Delta t}$. Then with the same initial state, the new sequence of hidden states is $h'_t = h_{t+\Delta t}$. The cell applies the same computation at every step, so shifting the input shifts the output by the same amount. This is a property of the *operator*: true at initialization, true after training, true for every weight setting. Weight sharing across time *is* time-translation equivariance.

(Strictly: the equivariance holds when the initial state h_0 is the same. In practice we reset $h_0 = 0$ between sequences, and that breaks the symmetry at sequence boundaries. The *within-sequence* computation is fully time-translation-equivariant, which is what matters for the inductive-bias claim.)

4. Unrolling and backprop through time

Training an RNN means computing the gradient of the loss with respect to W_{xh}, W_{hh}, and b_h for an unrolled sequence of length T. The standard technique is **backpropagation through time** (BPTT): unroll the cell T times, treat it as a deep feedforward network with T layers that happen to share weights, and apply the usual chain rule.

Forward: process $x_1, \ldots, x_T$ in order, threading the state $h_0 \to h_1 \to \ldots \to h_T$. At each step, an output layer (here a `Linear` projection to vocab logits, then `SoftmaxCrossEntropy`) gives a per-step loss L_t. The total loss is the sum:

$$L = \sum_{t=1}^{T} L_t.$$

Backward: the gradient on h_t has two sources:

$$\frac{\partial L}{\partial h_t} = \underbrace{\frac{\partial L_t}{\partial h_t}}_{\text{from } L_t \text{ directly}} + \underbrace{\frac{\partial L}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_t}}_{\text{from the future, through } h_{t+1}}.$$

We propagate backwards through time, summing the two contributions at each step. In code this is one extra argument on the cell's `backward`: on top of the gradient `dh_out` from the output projection, the cell also receives `dstate_next`, the gradient on h_t computed at the previous backward step (which is the step for time $t+1$). The two are added before applying the tanh derivative:

```
def backward(self, dh_out, dstate_next=None):
    x, h_prev, h_new = self.cache.pop()
    dh_total = [dh_out[i] + dstate_next[i] for i in range(H)]
    da = [dh_total[i] * (1.0 - h_new[i] ** 2) for i in range(H)]
    # accumulate dW_xh, dW_hh, db_h; backprop to dx and dh_prev.
    ...
    return dx, dh_prev
```

Weight gradients accumulate across all T timesteps. This is the same `+=` accumulation pattern the library has used since the foundations post's `Linear` layer: per-example gradients sum within a mini-batch; per-spatial-position gradients sum within a `Conv2D` forward (because the kernel is reused at every position); per-timestep gradients sum within an `RNNCell` unroll (because the recurrent matrix is reused at every timestep). Three flavors of the same operation, finer grain each time.

A gradient check verifies the math. Unroll the cell for $T = 3$ with a small `Linear` output projection and `SoftmaxCrossEntropy` loss; every parameter's analytical gradient matches central finite differences to 10^{-4} relative error (`tests/test_gradients.py::test_gradient_rnn_unrolled_bptt`).

5. Vanishing gradients

BPTT is correct but it is fragile. Each step of the backward pass multiplies the gradient on h_t by a Jacobian:

$$\frac{\partial h_{t+1}}{\partial h_t} \;=\; \operatorname{diag}\!\left(1 - h_{t+1}^2\right) W_{hh}.$$

Apply this T times in the backward pass, and the magnitude of the gradient is governed by the product of T such Jacobians, which in turn is governed by the spectral properties of W_{hh} (scaled by the tanh derivative, which is in $(0, 1]$ and almost always strictly less than 1). If the spectral radius of the effective Jacobian is below 1, the gradient on early timesteps decays exponentially in T: those parameters get essentially no signal from later losses. If the spectral radius is above 1, gradients blow up, and a single bad sequence can throw the weights off entirely.

This is the **vanishing-gradient problem** (and its evil twin, exploding gradients). It is the reason a vanilla RNN cannot easily learn dependencies more than a few dozen timesteps long. The architecture is the right shape for sequences in principle; the learning dynamics make long-range information transfer hard in practice.

Two standard mitigations:

- **Gated cells** (LSTM, GRU). Replace the simple tanh recurrence with an additive update that preserves a state across many timesteps. The LSTM's cell state c_t has Jacobian $\partial c_{t+1}/\partial c_t = \operatorname{diag}(f_t)$, where f_t is a learned forget gate with entries in $[0, 1]$. When the gate sits near 1, gradients flow through unattenuated. This is the "gradient highway" that makes LSTMs trainable on much longer sequences than vanilla RNNs.
- **Gradient clipping**. Cap the BPTT gradient norm at a threshold. Hacky but effective for exploding gradients.

This library implements vanilla `RNNCell` only. The extension to LSTM is mechanical: more matrix multiplies (one each for the input, forget, and output gates plus the cell-state update), the same `+=` accumulation pattern in backward, no new conceptual machinery. The pedagogical point is that the same Jacobian-product issue that vanilla RNNs suffer from is what motivated the whole gated-RNN literature, and what eventually motivated the Transformer.

6. The experiment: char-level Alice

We train a vanilla `RNNCell` plus a `Linear` output projection on the first 30,000 characters of Lewis Carroll's *Alice's Adventures in Wonderland*. Char-level: the model reads one character at a time, and its job at each step is to predict the next character given the state that summarizes everything it has read so far.

Configuration: `RNNCell(input_size=75, hidden_size=64)` plus `Linear(64, 75)` head, where 75 is the corpus's vocabulary (letters, digits, punctuation, whitespace). Sequence length 32 for BPTT unrolling. Learning rate 0.5 against the *mean* per-timestep gradient (gradients accumulate over the unroll, then `sgd_step` divides by T, exactly the convention `Network.step(lr, n)` uses for

mini-batches). Global gradient norm clipped to 5 (standard mitigation for exploding gradients in vanilla RNNs). One chunk per SGD step, 15 epochs. Total parameters:

$$64 \cdot 75 + 64 \cdot 64 + 64 + 75 \cdot 64 + 75 \;=\; 13{,}835.$$

Around 18 minutes of training in pure Python.

The mean per-character loss starts at $\log 75 \approx 4.32$ (uniform random over the vocabulary) and descends to about 2.06 over the 15 epochs. The interesting view is the *samples*: at each epoch boundary we generate 200 characters from the model, seeded with `"Alice "`. The samples track what the model has learned at each stage.

After 1 epoch (loss 3.09):

```
Alice he s ats ma,e
hingd, se d ait aoshinshe camtheing and phing anithithd ro toreth the
salb the til se  alobt the ta shel on ame the te phind the therscan
coe sh we rov thel
ind "ad, at wit on thad the a
```

After 5 epochs (loss 2.40):

```
Alice at yuirso rou hit coull at ant
ing us more, woothe, found le aQm aitid yor sheps woung teon the
forye soon tor, woy the the beme, bn more!
whes in toe heutle the
lomend won the walli,
"id sping the wa
```

After 10 epochs (loss 2.17):

```
Alice thed reand"
 _noudle to chate illitg, wh the; to hilg downd of noth" wand and eon
a keing the filk, the
"T le she wasser on t'e ond tale ad at who fat nowe me thay wast dome
to se soull gacd;! and th
```

After 15 epochs (loss 2.06):

```
Alice of seeply! be krear, bow and was riow. "I wond tele rit ine
halaid to was in, an beay of ere, and it hel  mowen-;ownd thit the
went veryon, our the Tabd mace poo sall aid tay miige tiree out ule
on en
```

The trajectory is the classic char-level RNN arc: random-character noise, then the *character distribution* (vowels and consonants alternate, spaces appear at plausible intervals), then *word-like fragments* (recognizable short words, plausible morphemes), then sentences that look locally English but drift in meaning across longer spans. With more training, a larger hidden state, longer sequences, and gated cells, the model would land on longer-range coherence. The vanilla RNN at modest scale and pure-Python compute lands short of that. The arc is the lesson; the absolute quality is the next post's territory.

The full demo is `examples/text_rnn.py`.

7. The pattern repeats

Three architectures, three different axes:

Architecture	Weight sharing axis	Equivariance
MLP	none	input permutation
CNN	spatial position	2-D translation in space
RNN	timestep	1-D translation in time

The pattern is the same. Identify the symmetry the data respects. Wire it into the architecture by reusing weights along the axis the symmetry runs along. The parameter count drops, the data efficiency rises, and the gradient backward becomes a `+=` accumulation along the shared axis.

What the RNN buys over a fixed-context CNN, for sequence data, is the *state*: information can in principle flow from arbitrarily distant past inputs through the hidden vector. What it costs is the vanishing-gradient problem of section 5, which limits how far that information actually transfers in practice.

8. Handoff to the Transformer

The next post takes the other path. Instead of carrying information forward through a sequential state, the **Transformer** lets every position attend to every other position directly, in a single layer. That move kills the vanishing-gradient problem (no long Jacobian products), restores parallelism over the sequence axis (no recurrence, so timesteps can be processed in any order), and exchanges time-translation equivariance for **permutation equivariance** over positions, which then has to be partially broken back with positional encoding so the model can still tell "the cat ate the mouse" from "the mouse ate the cat."

The Transformer is a different bet about which symmetries sequences respect. The RNN commits to time-translation equivariance and pays for it with sequential unrolling and vanishing gradients. The Transformer commits to permutation equivariance, breaks it explicitly with positional encoding, and pays for it with a parameter cost that scales quadratically in sequence length. Both are valid menu items on the inductive-bias menu, and the data picks the right one.